Informe de repaso y laboratorio de Media

1. Esquema general de Allin1

Allin1 es un marketplace con publicaciones de OFERTA y DEMANDA. El frontend permite al usuario interactuar; IAM gestiona autenticación, usuarios y perfiles; Catalog gestiona categorías y publicaciones; Media recibe, valida, procesa y almacena imágenes y sus referencias. Órdenes y chat están contemplados con diferente grado de implementación.

Frontend (Next.js) → API HTTP del microservicio correspondiente → lógica de servicio → repositorio / PostgreSQL. En Media, el archivo se envía a Cloudinary y su referencia se persiste en PostgreSQL mediante GORM. No todas las llamadas atraviesan gRPC: Media expone handlers HTTP directos en el puerto 8083.

2. Recorrido completo: imagen de perfil

1. La página de prueba crea un FormData con tipo=PERFIL y file=<imagen>.
2. Envía POST http://localhost:8083/v1/media/upload con Authorization: Bearer <JWT>.
3. El middleware HTTP verifica el JWT y coloca el identificador del usuario en el contexto.
4. El handler comprueba método POST y exactamente un archivo.
5. El servicio Media comprueba tipo y repositorio; el pipeline lee, valida formato/tamaño y procesa la imagen.
6. El uploader envía el resultado a Cloudinary.
7. GORM guarda usuario, tipo, URL y public_id en PostgreSQL.
8. El endpoint responde HTTP 201 con id, tipo, url y publicId; la página muestra la URL.

Límite de integración: cargar la imagen no actualiza automáticamente los campos de perfil en IAM. La actualización de perfil es una operación separada (PUT /v1/auth/profile). Referencias para revisar en el repositorio:

- **Page.tsx : 28**
- **main.go : 52**
- **Jwt_http.go : 17**
- **Media_handler.go : 32**
- **Media_service.go : 59**
- **Auth.proto: 35**

3. Aportes y trazabilidad

Los siguientes commits están relacionados con Media;

Commit	Cambio	Participación personal
23eff8f	Endpoint multipart y Cloudinary	Si
9a9c1cf	Detección de formato por magic bytes	Si
0d976a0	Redimensionamiento y compresión	Si
8df1c88	Pruebas del pipeline	Si
b02b306	Endpoint JWT y persistencia	Si
ecf0927	Dimensiones por PERFIL/PORTADA	Si

4. Simulacro individual: 12 preguntas y respuestas de referencia

1. [Conceptos] ¿Qué corresponde a Media y qué a IAM?

Respuesta: Media procesa y sube imágenes, y registra referencias; IAM gestiona identidad y perfiles. Subir no modifica automáticamente el perfil.

Evidencia para cotejar: media_service.go:59; auth_service.go:336

2. [Conceptos] ¿Qué diferencia hay entre validar y procesar?

Respuesta: Validar determina si un archivo se acepta por contenido, formato y tamaño. Procesar transforma imágenes aceptadas, por ejemplo redimensionándolas.

Evidencia para cotejar: validator.go:73; processor.go:65

3. [Conceptos] ¿Para qué se usan Cloudinary, PostgreSQL y GORM?

Respuesta: Cloudinary almacena el archivo; PostgreSQL conserva la referencia y asociación con usuario/tipo; GORM realiza la persistencia.

Evidencia para cotejar: upload_service.go:95; media_service.go:43; media.go:13

4. [Lectura de código] ¿Un texto renombrado a .jpg pasa la validación?

Respuesta: No: se inspecciona la firma real del contenido; el test espera 415 y cero subidas.

Evidencia para cotejar: validator.go:84; upload_handler_test.go:176

5. [Lectura de código] ¿Qué límites y formatos existen?

Respuesta: JPEG, PNG, WEBP y AVIF; validador general de 5 MiB y 10 archivos por lote. Perfil/portada: un archivo; procesador general 1920 px, perfil 512 y portada 1600.

Evidencia para cotejar: validator.go:7; media_handler.go:57; processor.go:14; media_service.go:27

6. [Lectura de código] ¿Cómo obtiene el servicio el ID del usuario?

Respuesta: El middleware extrae y valida el JWT y coloca el ID en contexto; el formulario no decide el propietario. La ruta /v1/media/upload está protegida.

Evidencia para cotejar: jwt_http.go:17; media_handler.go:38; main.go:51

7. [Integración] ¿Cuál es el flujo de subida desde el frontend?

Respuesta: FormData y Bearer → middleware → handler → validación/proceso → Cloudinary → registro en BD → HTTP 201 con referencia.

Evidencia para cotejar: page.tsx:28; media_handler.go:63

8. [Integración] ¿Cómo se incorpora la URL al perfil?

Respuesta: La subida retorna una URL; IAM tiene un PUT /v1/auth/profile separado.

Evidencia para cotejar: auth.proto:35,99; page.tsx:32

9. [Integración] ¿Media usa el mismo recorrido gRPC que IAM y está desplegado en Kubernetes?

Respuesta: Media recibe HTTP directo en 8083. No se identificó ruta central de Nginx ni Deployment/Service de Media en el estado revisado.

Evidencia para cotejar: main.go:46; configmap.yaml:232; pvc.yaml:26

10. [Decisiones y fallos] ¿Qué sucede si Cloudinary falla?

Respuesta: El error llega al handler; la prueba del uploader simulado espera HTTP 500, sin llamar al servicio externo real.

Evidencia para cotejar: upload_handler_test.go:266; upload_handler.go:87

11. [Decisiones y fallos] ¿Qué ocurre si Cloudinary sube pero falla la BD?

Respuesta: Puede quedar una imagen en Cloudinary sin referencia persistida; no se encontró compensación automática en la operación revisada.

Evidencia para cotejar: media_service.go:84

12. [Decisiones y fallos] ¿Todos los formatos se redimensionan igual?

Respuesta: No: AVIF puede pasar sin transformación. La recodificación a JPEG de otros formatos puede perder transparencia.

Evidencia para cotejar: types.go:17; processor.go:65; processor_test.go:224

4. Laboratorio: cinco casos de prueba

Se ejecutaron cinco pruebas automatizadas del microservicio Media utilizando el comando go test desde la carpeta backend-go. Todas finalizaron exitosamente, con un resultado de **5 pruebas aprobadas de 5 ejecutadas**.

L1 — Subida válida

- Entrada: dos imágenes JPEG de 100×100 y 4000×3000.
- Esperado: HTTP 200, dos subidas simuladas y dos URLs.
- Observado: TestHandle_SubidaExitosa — PASS (1,62 s).
- Evidencia: salida de la terminal con === RUN, --- PASS y PASS.

L2 — Formato inválido

- Entrada: archivo de texto con extensión .jpg.
- Esperado: HTTP 415 y cero llamadas al uploader.
- Observado: TestHandle_FormatoInvalido — PASS (0,00 s).
- Evidencia: salida de la terminal con --- PASS.

L3 — Archivo demasiado grande

- Entrada: 4000 bytes con un límite de prueba de 1000 bytes.
- Esperado: HTTP 413 y cero llamadas al uploader.
- Observado: TestHandle_ArchivoIndividualDemasiadoGrande — PASS (0,00 s).
- Evidencia: salida de la terminal con --- PASS.

L4 — Redimensionamiento

- Entrada: imagen JPEG de 4000×3000 píxeles.
- Esperado: imagen procesada de 1920×1440 píxeles.
- Observado: TestProcessRedimensionaCuandoExcedeElLimite — PASS (1,82 s).
- Evidencia: salida de la terminal con --- PASS.

L5 — Fallo externo

- Entrada: imagen JPEG de 50×50 y uploader simulado que devuelve un error.
- Esperado: HTTP 500.
- Observado: TestHandle_FalloDeCloudinary — PASS (0,00 s).
- Evidencia: salida de la terminal con --- PASS.

Conclusión del laboratorio

Los cinco casos de prueba se ejecutaron y aprobaron correctamente. Los resultados respaldan el comportamiento esperado de los componentes evaluados ante subidas válidas, archivos inválidos, límites de tamaño, redimensionamiento y fallos simulados del almacenamiento externo.

Las pruebas utilizan componentes simulados y no demuestran por sí solas una integración completa con Cloudinary real, PostgreSQL, JWT o IAM. Por lo tanto, se considera completado el laboratorio de pruebas aisladas, manteniendo esas integraciones como aspectos no comprobados.

5. Contraste de contratos y consumidor frontend

Aspecto	Subida múltiple	Perfil / portada
Ruta	POST /upload/multiple	POST /v1/media/upload
Multipart	files (varios)	tipo + file (uno)
JWT	Sin middleware en ruta revisada	Bearer JWT requerido
Éxito	200; success + files	201; id, tipo, url, publicId
Persistencia	Subida a Cloudinary	Cloudinary + PostgreSQL
Cobertura L1–L5	Cuatro pruebas de handler	No cobertura integral

La página de prueba del frontend consume el contrato de perfil/portada y muestra la URL. No se debe interpretar que el laboratorio demuestra la actualización real del perfil en IAM.

Referencias: main.go:51

upload_handler.go:46;

media_handler.go:55;

media_service.go:78;

page.tsx:28.